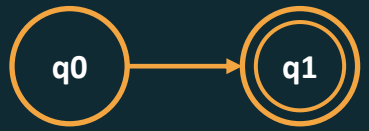
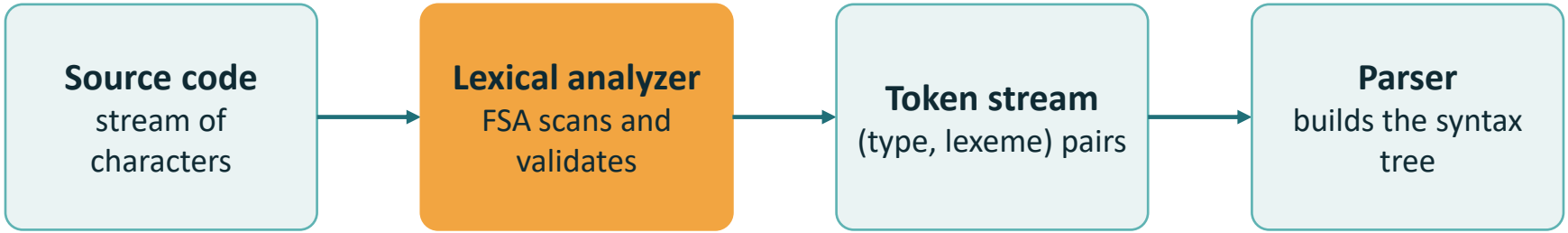
Building a Finite State Automaton for Token Recognition

Focus: Lexical Analysis, Token Validation

Presented by Obul
CSA0639 – Design and Analysis of Algorithms for Complexity Problems
Saveetha Institute of Medical and Technical Sciences (SIMATS)



Lexical Analysis in the Compiler



Input line: `int count = 42;`

KEYWORD	IDENTIFIER	OPERATOR	NUMBER	DELIMITER
`int`	`count`	`=`	`42`	`;`

Anything that matches no token pattern is reported as a lexical error.

Tokens, Lexemes and Patterns

Token	Pattern	Example lexemes
Identifier	`[A-Za-z_][A-Za-z0-9_]*`	count, _tmp1, x9
Integer	`[0-9]+`	42, 007
Float	`[0-9]+\.[0-9]+`	3.14, 0.5
Operator	`+ | - | * | / | = | ==`	=, ==
Delimiter	`; | , | ( | ) | { | }`	(, ;

A token is the category, a lexeme is the matched text, and a pattern describes which lexemes belong. Keywords match the identifier pattern, so a reserved-word lookup follows the ID automaton.

Finite Automaton Fundamentals

- **Q** — Finite set of states
- **Σ** — Input alphabet: letters, digits, symbols
- **δ** — Transition function, $Q \times \Sigma \rightarrow Q$
- **q0** — Start state
- **F** — Set of accepting (final) states

Why a DFA for lexing?

- Exactly one move per input character
- Fast: a table lookup, no backtracking
- Regex to NFA to DFA via subset construction

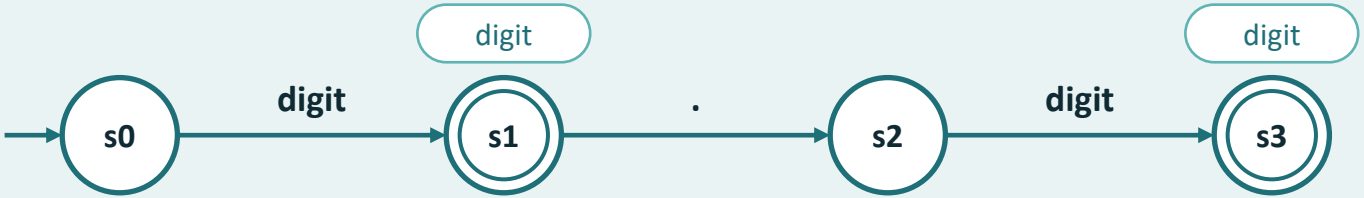
Designing the Token FSAs

Identifier



Accepts: count, _tmp1, x9
Rejects: 9x, @id

Number



Token Validation Algorithm

Transition table: identifier DFA

State	letter, _	digit	other
→ **q0**	q1	dead	dead
* **q1**	q1	q1	dead

→ start state * accepting state

```
state = q0
for ch in lexeme:
  state = T[state][cls(ch)]
  if state == dead: return False
return state in F
```

Maximal munch

Keep consuming characters while a transition exists, then emit the longest valid token.

Keyword check

After the ID automaton accepts, look the lexeme up in a reserved-word table.

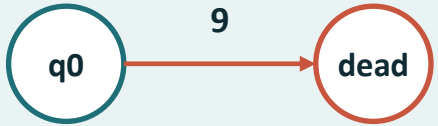
Worked Example and Complexity

Input: x9 — ACCEPT



Ends in accepting state q1

Input: 9x — REJECT



No digit move from q0, so reject at once

O(n) time per lexeme of length n

O(1) work per character (one lookup)

|Q|·|Σ| transition table space

Thank You

Questions and discussion

Obul
CSA0639 – Design and Analysis of Algorithms for Complexity Problems
SIMATS

